2026-04-22 10:35:27 **You Asked**

pim_pam_boing_eccomerce.js

tengo una duda tengo que hacer un como mdocumento en navicat con mongo db pero en el ejemplo que me dieron hizimos uns collecciones llamadas promociones, tikets y otra pero debe de estar en base ah mi videojuego que sobre tda pero no se que hacer ahi, osea promociones oh como que mas tablas debo de agregar?

Gemini 3 Flash

¡Entiendo perfectamente! Estás pasando de un ejemplo genérico (promociones y tickets) a algo que sea específico para tu videojuego de **Tower Defense Académico (TDA)**.

En un juego de Tower Defense, la estructura de datos en MongoDB debe reflejar lo que el jugador posee, lo que puede comprar y cómo progresa. Como estás usando **Navicat con MongoDB**, lo ideal es que pienses en "Colecciones" como si fueran las piezas del motor de tu juego.

Aquí te doy sugerencias de qué colecciones (tablas) deberías agregar para que tu proyecto de TDA sea profesional y funcional:

---

1. Colección: `torres` (Tu "Catálogo")

En lugar de solo "promociones", necesitas definir qué torres existen en tu juego y qué características tienen.

- **¿Qué guardar aquí?**: Nombre de la torre, daño base, costo de construcción, velocidad de ataque y tipo de daño (fuego, hielo, físico).
- **Ejemplo de documento**:

```json
{
  "clave": "torre_laser_01",
  "nombre": "Torre de Cristal",
  "daño": 50,
  "rango": 5,
  "costo_oro": 150,
```

```
  "descripcion": "Dispara un rayo continuo que aumenta el daño con el tiempo."
}
```

2. Colección: `niveles` o `mapas`

Para un TDA, necesitas saber qué retos enfrentará el usuario.

- **¿Qué guardar aquí?**: Nombre del mapa, dificultad, número de oleadas (waves) y la recompensa por completarlo.
- **Ejemplo**: `mapa_biblioteca_central`, `dificultad: "Media"`, `recompensa_xp: 500`.

3. Colección: `progreso_usuario`

Esta es la más importante. Relaciona al usuario con lo que ha logrado en el juego.

- **¿Qué guardar aquí?**: ID del usuario, nivel actual, experiencia acumulada (XP), monedas premium (si tiene) y una lista de torres que ya desbloqueó.
- **Ejemplo**:

```json
{
  "usuario_id": 32,
  "nivel_cuenta": 10,
  "monedas_oro": 2500,
  "torres_desbloqueadas": ["torre_basica", "torre_laser_01"]
}
```

4. Colección: `enemigos` (Opcional pero recomendado)

Si quieres que tu base de datos sea completa, puedes tener un catálogo de los "villanos" o errores académicos que atacan la torre.

- **¿Qué guardar aquí?**: Nombre del enemigo, vida (HP), velocidad de movimiento y cuántas monedas suelta al morir.

¿Cómo adaptarlo en tu script de Navicat?

Para que coincida con el estilo de tu archivo `pim_pam_boing_eccomerce.js`, deberías estructurarlo así (puedes copiar este ejemplo para tu nueva colección):

```javascript
// ----------------------------
// Collection structure for torres
// ----------------------------
```

```
db.getCollection("torres").drop();
db.createCollection("torres");

// ----------------------------
// Documents of torres
// ----------------------------
db.getCollection("torres").insert([ {
    clave: "torre_quimica_01",
    nombre: "Lanza Ácido",
    descripcion: "Torre académica que lanza conceptos químicos que corroen la armadura.",
    daño: 20,
    rango: 10,
    costo: 200
} ]);
```

Resumen de lo que te sugiero hacer:

1. **Mantén** `promociones` : Úsala para vender "Packs de bienvenida" de torres raras.
2. **Mantén** `ticket_soporte` : Por si a un jugador se le traba el juego.
3. **AGREGA** `torres` : Es el corazón de tu Tower Defense.
4. **AGREGA** `inventario_jugador` : Para saber qué tiene cada quién.

¿Te gustaría que te ayude a redactar el código de alguna de estas colecciones específicas para que solo lo pegues en Navicat?

2026-04-22 10:39:12 **You Asked**

solo es un videojuego muy basico donde entras al juego ingresas escribes tu nombre despues te lleva donde estan tus niveles, que son 4 el primero es de un memorama, el segundo es seguir la secuencia de la cancion, la tercera es de un nivel donde sale un bote de basura y tienes que recoger la basura que va cayebndo por el aire, luego el ultimo se trata de un conejo que debes de ayudar pasar la carretera

Gemini 3 Flash

¡Entendido! Al ser un juego basado en niveles con mecánicas tipo "minijuegos", tu base de datos debe enfocarse en registrar el **progreso** del usuario en esos desafíos específicos y los **objetos** que interactúan en ellos.

Como ya tienes la estructura de `promociones` y `ticket_soporte`, aquí te presento las colecciones que le darían sentido a tu **Tower Defense Académico** (o juego de niveles):

---

1. Colección: `usuarios`

Dado que el flujo inicia cuando el jugador **escribe su nombre**, necesitas guardar ese registro para asociarlo a sus puntajes.

- **Campos sugeridos**: `nombre_usuario`, `fecha_registro`, `nivel_maximo_alcanzado`.
- **Ejemplo de documento**:

```javascript
{
  "nombre_usuario": "Juanito99",
  "nivel_actual": 1,
  "ultimo_acceso": "2026-04-22 10:00:00"
}
```

2. Colección: `niveles_logros`

Esta colección reemplaza la idea de "inventario" y se enfoca en cómo le fue al usuario en los 4 niveles que mencionas.

Nivel	Tipo de Juego	Datos a guardar
Nivel 1	Memorama	Tiempo récord y número de intentos.
Nivel 2	Secuencia Musical	Notas acertadas y racha (combo).
Nivel 3	Recolección (Basura)	Cantidad de basura atrapada.
Nivel 4	El Conejo	Vidas restantes y tiempo de cruce.

3. Colección: `catalogo_niveles` (Estructura de Datos)

Aquí defines las reglas de cada nivel, similar a cómo se definen las `promociones` en tu archivo original.

```javascript
db.getCollection("catalogo_niveles").insert([
  {
    "id_nivel": 1,
    "nombre": "Memorama Académico",
    "objetivo": "Emparejar todas las cartas antes de que acabe el tiempo",
```

```
    "dificultad": "Fácil"
  },
  {
    "id_nivel": 3,
    "nombre": "Limpieza Aérea",
    "objetivo": "Atrapa 20 objetos con el bote de basura",
    "puntos_por_objeto": 10
  }
]);
```

4. Colección: `ranking_global`

Para que el juego sea más interesante, puedes guardar los mejores puntajes de todos los usuarios.

¿Cómo agregarlo a tu script de Navicat?

Basado en el formato de tu archivo de **MongoDB 8.2.3**, podrías añadir una sección de "**Estructura para el progreso del juego**" así:

```javascript
// ----------------------------
// Collection structure for progreso_niveles
// ----------------------------
db.getCollection("progreso_niveles").drop();
db.createCollection("progreso_niveles");

// Ejemplo de inserción para el nivel del Conejo
db.getCollection("progreso_niveles").insert([ {
    usuario: "NombreDelJugador",
    nivel: 4,
    nombre_juego: "Cruce del Conejo",
    puntaje_maximo: 500,
    completado: true,
    fecha_logro: "2026-04-22"
} ]);
```

Sugerencia adicional: Ya que tienes una colección de `promociones` que incluye "recursos digitales" y "beneficios para subir de nivel", podrías hacer que si el usuario compra una "promoción", el nivel del **Bote de Basura** sea más grande o el **Conejo** tenga más vidas.

¿Te gustaría que te ayude a redactar el código de validación (`validator`) para alguna de estas nuevas tablas?

2026-04-22 10:41:45 **You Asked**

me dijeron que si debo de agregarle promociones y tikect aunque en si el videojuego no lo tenga

Gemini 3 Flash

Entiendo perfectamente la situación. A veces en los proyectos escolares o técnicos te piden agregar funciones como "**Promociones**" y "**Tickets**" aunque el juego sea básico, simplemente para que demuestres que sabes manejar diferentes tipos de datos (compras, arreglos, objetos complejos y validaciones) en MongoDB.

Para que tu proyecto de **Navicat** tenga sentido y no se vea "pegado", vamos a adaptar esas colecciones para que se sientan parte de tu juego de los 4 niveles.

Aquí te digo cómo justificar y estructurar esas colecciones:

---

1. Colección: `promociones` (Justificación: Tienda de Power-ups)

Aunque tu juego sea gratuito, puedes simular una tienda donde el usuario "compra" ayudas para los niveles difíciles (como el del conejo o la secuencia musical).

- **¿Qué vender?**: "Vidas extra para el conejo", "Cámara lenta para la basura", "Pista para el memorama".
- **Estructura sugerida**:

```javascript
db.getCollection("promociones").insert([ {
    clave: "pack_super_conejo",
    nombre: "Kit de Supervivencia - Nivel 4",
    descripcion: "Incluye 3 vidas extra y un escudo protector para cruzar la carretera.",
    costo: 50, // Monedas del juego
    estatus: "Activa"
} ]);
```

2. Colección: `ticket_soporte` (Justificación: Reporte de Errores)

Esta es muy útil para demostrar que sabes usar **Validadores de Esquema** (como el que viene en tu archivo original). En un juego, los tickets sirven para que el usuario reporte si un nivel no carga o si se trabó el conejo.

- **¿Qué reportar?**: "El nivel 3 no detecta cuando atrapo la basura", "Se perdió mi puntaje del memorama".
- **Estructura sugerida**:

```javascript
db.getCollection("ticket_soporte").insert([ {
    clave: "ticket_004",
    usuario: "Jugador_Pro",
    nivel_afectado: 3,
    tipo_reporte: "Bug de colisión",
    titulo: "La basura atraviesa el bote",
    estatus: "Pendiente"
} ]);
```

---

3. La nueva colección clave: `niveles`

Para que tu documento esté completo, te sugiero agregar esta colección que conecte todo. Aquí definirás los 4 juegos que mencionaste:

Nivel	Nombre	Mecánica principal
1	Memorama	Emparejar cartas (Lógica)
2	Ritmo Musical	Seguir secuencias (Memoria auditiva)
3	Recolección	Atrapar basura al vuelo (Reflejos)
4	El Conejo	Cruce de carretera (Timing)

Ejemplo de código para Navicat:

```javascript
// ---------------------------
// Estructura para los minijuegos
// ---------------------------
db.getCollection("configuracion_niveles").drop();
db.createCollection("configuracion_niveles");

db.getCollection("configuracion_niveles").insert([
  {
    n_nivel: 1,
    nombre: "Memorama de Elementos",
    dificultad: "Fácil",
    tiempo_limite_seg: 60
```

```
  },
  {
    n_nivel: 4,
    nombre: "El Conejo Aventurero",
    dificultad: "Difícil",
    vidas_iniciales: 3
  }
]);
```

¿Por qué hacerlo así?

1. **Cumples con la tarea**: Agregas `promociones` y `tickets` como te pidieron.
2. **Le das coherencia**: Las promociones sirven para los niveles y los tickets para reportar fallas en ellos.
3. **Demuestras nivel**: Usas `ObjectId`, `strings`, `integers` y `booleanos`, que es lo que buscan evaluar en un dump de MongoDB.

¿Quieres que te ayude a armar el código final de alguna de estas para que solo lo copies y pegues en tu archivo `.js`?

2026-04-22 10:42:45 **You Asked**

y tambien validaaciones

Gemini 3 Flash

¡Excelente decisión! Las **validaciones** (JSON Schema) son las que realmente demuestran que dominas MongoDB, ya que obligan a que los datos sigan reglas estrictas, evitando que alguien escriba un nombre donde va un número o deje campos vacíos.

Para tu juego de los 4 niveles, vamos a crear una estructura profesional que incluya las validaciones para los **Usuarios** y sus **Puntajes**.

Aquí tienes el código que puedes agregar a tu archivo `.js` para implementar estas colecciones con sus respectivas reglas:

---

1. Validación para la Colección `usuarios`

Esta regla asegura que cada jugador tenga un nombre obligatorio y que su nivel actual sea un número entre 1 y 4.

```javascript
// ----------------------------
// Estructura y Validación para usuarios
// ----------------------------
db.getCollection("usuarios").drop();
db.createCollection("usuarios", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["nombre_jugador", "nivel_actual", "fecha_registro"],
            properties: {
                nombre_jugador: {
                    bsonType: "string",
                    description: "Debe ser un texto y es obligatorio"
                },
                nivel_actual: {
                    bsonType: "int",
                    minimum: 1,
                    maximum: 4,
                    description: "Solo puede ser un número del 1 al 4"
                },
                fecha_registro: {
                    bsonType: "date",
                    description: "Fecha en la que el usuario ingresó por primera vez"
                }
            }
        }
    }
});
```

---

2. Validación para la Colección `sesiones_juego`

Esta es la más importante para tu proyecto. Aquí validamos los resultados de los 4 minijuegos (Memorama, Secuencia, Basura y Conejo).

```javascript
// ----------------------------
// Estructura y Validación para sesiones_juego
// ----------------------------
db.getCollection("sesiones_juego").drop();
db.createCollection("sesiones_juego", {
```

```
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["usuario_id", "tipo_nivel", "puntaje_obtenido"],
            properties: {
                usuario_id: {
                    bsonType: "objectId",
                    description: "Referencia al ID del usuario"
                },
                tipo_nivel: {
                    enum: ["memorama", "secuencia_musical", "recoleccion_basura",
"cruce_conejo"],
                    description: "Solo puede ser uno de los 4 nombres de tus niveles"
                },
                puntaje_obtenido: {
                    bsonType: "int",
                    minimum: 0,
                    description: "El puntaje no puede ser negativo"
                },
                detalles_tecnicos: {
                    bsonType: "object",
                    properties: {
                        tiempo_segundos: { bsonType: "int" },
                        vidas_restantes: { bsonType: "int" }
                    }
                }
            }
        }
    }
});
```

---

3. Ejemplo de Documentos (Inserts)

Para que tu base de datos no esté vacía al entregarla, agrega estos ejemplos que respetan las validaciones anteriores:

```javascript
// Insertar un usuario
db.getCollection("usuarios").insert({
    nombre_jugador: "ProGamer_TDA",
    nivel_actual: 1,
    fecha_registro: new Date()
});

// Insertar un resultado del Nivel 4 (El Conejo)
```

```
db.getCollection("sesiones_juego").insert({
    usuario_id: ObjectId("69e11c16c4b0f4b1f6088267"), // Usa un ID de tus usuarios
    tipo_nivel: "cruce_conejo",
    puntaje_obtenido: 850,
    detalles_tecnicos: {
        tiempo_segundos: 45,
        vidas_restantes: 2
    }
});
```

¿Por qué esto ayuda a tu proyecto?

- **Seguridad**: Si alguien intenta poner un nivel "5", MongoDB lanzará un error porque tu validación dice que el máximo es **4**.
- **Orden**: Al usar `enum` en `tipo_nivel`, garantizas que nadie escriba mal los nombres de los juegos (ej. escribir "conejo" en lugar de "cruce_conejo").
- **Coherencia**: Mantienes el mismo estilo técnico que el archivo original que subiste (con `bsonType` y `description`).

¿Necesitas que ajuste alguna regla de validación, como por ejemplo que el nombre del jugador tenga un mínimo de letras?

2026-04-22 10:43:36 **You Asked**

entonces cuales serian ls colecciones esenciales para mi videojuego?

Gemini 3 Flash

Para que tu videojuego de los 4 niveles (Memorama, Secuencia Musical, Recolección de Basura y El Conejo) esté completo y cumpla con lo que te piden, estas son las **5 colecciones esenciales** que debes tener en tu base de datos de MongoDB:

1. `usuarios` (El Corazón del Juego)

Es fundamental para registrar quién está jugando. Como mencionaste que el usuario escribe su nombre al entrar, esta colección captura ese momento.

- **Campos clave:** `nombre_jugador`, `nivel_progreso` (del 1 al 4), `fecha_registro`.
- **Propósito:** Guardar el estado actual de cada jugador y su identidad.

2. `sesiones_juego` (Tus 4 Niveles)

Aquí es donde guardas los resultados de cada partida. Es la colección que conecta con tus 4 mecánicas.

- **Campos clave:** `usuario_id`, `tipo_nivel` (Memorama, Secuencia, Basura o Conejo), `puntaje_obtenido`, `tiempo_empleado`.
- **Propósito:** Historial de desempeño. Sirve para saber si el usuario ya pasó el nivel del Memorama y puede avanzar al del Conejo.

3. `promociones` (Requisito Académico / Tienda)

Aunque tu juego sea básico, esta colección simula la parte comercial o de "ventas" que te solicitaron incluir.

- **Campos clave:** `clave`, `nombre` (ej. "Pase Saltador para Conejo"), `costo`, `descripcion`.
- **Propósito:** Ofrecer "ventajas" para los niveles, como vidas extra o más tiempo para el memorama.

4. `ticket_soporte` (Atención al Cliente)

Esta colección es esencial para demostrar que sabes manejar **validaciones complejas** y esquemas con objetos anidados.

- **Campos clave:** `clave`, `usuario_id`, `tipo_reporte` (ej. "Falla en nivel 3"), `estatus` (Abierto/Cerrado).
- **Propósito:** Permitir que los jugadores reporten errores, como si el conejo se queda trabado en la carretera.

5. `catalogo_niveles` (Configuración)

En lugar de escribir las reglas del juego en el código, las guardas aquí.

- **Campos clave:** `numero_nivel`, `nombre_juego`, `objetivo_puntos`, `dificultad`.
- **Propósito:** Definir los parámetros de los 4 niveles (ej. cuánta basura debe caer en el nivel 3 para ganar).

Resumen de la Estructura en Navicat

Si juntas todo, tu base de datos `pim_pam_boing_eccomerce` (o el nombre de tu juego) se vería así en el explorador de Navicat:

Colección	¿Para qué sirve en tu juego?
`usuarios`	Registra el nombre que escriben al inicio.

Colección	¿Para qué sirve en tu juego?
sesiones_juego	Guarda los récords de los 4 niveles.
promociones	Vende "Power-ups" para facilitar los juegos.
ticket_soporte	Reporta si un minijuego falla.
valoraciones	(Opcional) Para que el usuario diga si le gustaron los niveles.

¿Te gustaría que te ayude a generar el código de **validación** para la colección de `usuarios` para que nadie pueda dejar su nombre vacío?

2026-04-23 19:18:12 **You Asked**

una pregunta si yo quiero decargar un documento de un grupo en git hub, ah ese grupo le llega la notificasion que descargue su documento?

Gemini 3 Flash

Esa es una duda muy común cuando uno está empezando a usar GitHub. La respuesta corta es: **No, no les llega ninguna notificación.**

Aquí te explico los detalles de qué pueden ver y qué no:

---

1. Descargar el archivo (Download ZIP)

Si entras al repositorio, haces clic en el botón verde **"Code"** y luego en **"Download ZIP"**, GitHub **no avisa a nadie**. No hay una campana de notificación ni un correo que les diga que tú descargaste el archivo.

2. Clonar el repositorio (`git clone`)

Si usas la terminal para clonar el proyecto a tu computadora, tampoco se genera una notificación. Los dueños del grupo pueden ver un gráfico de "tráfico" que indica que hubo una descarga, pero **no les dice quién fue** (es anónimo).

3. Cuando SÍ les llega notificación

Para que a un grupo de GitHub le llegue una notificación sobre tu actividad, tendrías que interactuar de forma "pública" con el código, por ejemplo:

- **Hacer un Fork:** Crear una copia del proyecto en tu propia cuenta.
- **Dar una Star:** Darle "like" al proyecto.
- **Abrir un Issue o Pull Request:** Si comentas algo o intentas subir un cambio.
- **Seguirlos:** Si le das "Follow" a la organización o a un miembro.

En resumen:

Si solo entras y descargas el documento o el código para estudiarlo o usarlo en tu proyecto de **Navicat**, puedes estar tranquilo: **eres invisible para ellos.** ¡Nadie sabrá que bajaste el archivo a menos que tú mismo les digas o hagas una interacción pública!

2026-04-23 19:24:19 **You Asked**

MongoDB.pdf

tengo que hacer un documento asi pero en base ah mi videojuego ayudame ah realizarla

Gemini 3 Flash

Para realizar tu documento siguiendo el ejemplo del PDF pero adaptado a tu videojuego de los 4 niveles (Memorama, Secuencia, Basura y Conejo), debemos estructurarlo de manera formal.

Aquí tienes una propuesta completa y profesional que puedes usar para tu entrega. [cite_start]Este documento combina la estructura que subiste en el PDF [cite: 1, 2, 3] con las necesidades de tu juego:

Esquema NoSQL (MongoDB) - Proyecto Videojuego TDA

[cite_start]Como complemento al modelo relacional, se diseñó un esquema NoSQL orientado al análisis del progreso del jugador y la flexibilidad de datos de los minijuegos[cite: 2]. [cite_start]Se implementaron las colecciones clave solicitadas, adaptadas a la mecánica de niveles del proyecto[cite: 3].

1. Colección: `usuarios`

Esta colección registra la identidad del jugador al ingresar su nombre y su avance general en el juego.

```json
{
  "_id": ObjectId("69e11c16c4b0f4b1f6088260"),
  "nombre_jugador": "EcoPlayer_26",
  "nivel_progreso": 3,
  "fecha_registro": "2026-04-23",
  "ultimo_acceso": "2026-04-23T10:30:00Z"
}
```

2. Colección: `sesiones_juego` (Resultados de Niveles)

Aquí se almacenan los puntajes y el desempeño en cada uno de los 4 minijuegos.

```json
{
  "_id": "sesion_001",
  "id_usuario": ObjectId("69e11c16c4b0f4b1f6088260"),
  "tipo_nivel": "cruce_conejo",
  "puntaje_obtenido": 850,
  "detalles": {
    "tiempo_segundos": 42,
    "vidas_restantes": 2,
    "completado": true
  },
  "fecha_partida": "2026-04-23"
}
```

3. Colección: `valoraciones`

[cite_start]Permite a los jugadores calificar su experiencia en los niveles educativos[cite: 4].

```json
{
  "_id": "val_001",
  "id_usuario": 12,
  "nickname": "EcoPlayer",
  "calificacion": 5,
  "comentario": "El nivel del bote de basura es muy divertido y educativo",
  "fecha": "2026-04-19",
  "nivel_alcanzado": 4
}
```

[cite_start]*[Cita: Estructura basada en reporte de valoraciones [cite: 7, 8, 9, 10, 11, 12, 13]]*

4. Colección: `ticket_soporte`

[cite_start]Se utiliza para reportar fallas técnicas en los minijuegos, como errores de carga en el nivel 3[cite: 18, 21].

```json
{
  "_id": "ticket_001",
  "id_usuario": 12,
  "tipo": "soporte",
  "descripcion": "Error al cargar nivel 3 (Recolección de basura)",
  "estado": "abierto",
  "prioridad": "media",
  "fecha_creacion": "2026-04-19",
  "fecha_resolucion": null
}
```

[cite_start]*[Cita: Estructura basada en reporte de tickets [cite: 16, 17, 18, 22, 23, 24, 25]]*

5. Colección: `promociones`

[cite_start]Define los beneficios o descuentos activos para el contenido del juego[cite: 27, 28].

```json
{
  "_id": "promo_001",
  "nombre": "Pack Ayuda Conejo",
  "descripcion": "3 vidas extra para el nivel 4 del conejo",
  "tipo": "beneficio_juego",
  "valor": 3,
  "fecha_inicio": "2026-04-01",
  "fecha_fin": "2026-04-30",
  "activo": true
}
```

[cite_start]*[Cita: Estructura basada en catálogo de promociones [cite: 27, 28, 29, 32, 33, 34]]*

Notas Técnicas para tu entrega:

1. [cite_start]**Validación**: Cada colección incluye tipos de datos específicos (`ObjectId` , `string` , `int` , `boolean`) para asegurar la integridad[cite: 1, 2].
2. [cite_start]**Relación**: El campo `id_usuario` actúa como el vínculo entre el progreso del juego y el perfil del jugador[cite: 8, 17].

2026-04-23 19:34:47 **You Asked**

pero yo no quiero de eso si no de tickes, promociones y validaciones

Gemini 3 Flash

Entiendo perfectamente. Quieres que el documento se centre exclusivamente en las tres colecciones que te solicitaron (**Tickets, Promociones y Valoraciones**) pero adaptadas 100% a las mecánicas de tu juego (Memorama, Secuencia, Basura y Conejo).

Aquí tienes el documento estructurado tal como el ejemplo de tu PDF:

Esquema NoSQL (MongoDB) - Gestión de Soporte y Usuario

[cite_start]Como complemento al modelo relacional, se diseñó un esquema NoSQL orientado al análisis de retroalimentación y resolución de incidencias en los minijuegos[cite: 2]. Se implementaron las tres colecciones clave solicitadas:

1. Colección: `valoraciones`

Esta colección permite medir la satisfacción del usuario en cada uno de los 4 niveles educativos.

```json
{
  "_id": "val_001",
  "id_usuario": 12,
  "nickname": "EcoPlayer",
  "calificacion": 5,
  "comentario": "El nivel del bote de basura es muy divertido y educativo",
  "fecha": "2026-04-19",
  "nivel_alcanzado": 3
}
```

- [cite_start]**_id**: Identificador único de la reseña[cite: 7].
- [cite_start]**nickname**: Nombre que el usuario ingresó al iniciar el juego[cite: 9].
- [cite_start]**calificacion**: Puntuación del 1 al 5 otorgada al minijuego[cite: 10].
- [cite_start]**nivel_alcanzado**: Indica en cuál de los 4 niveles se encontraba el jugador[cite: 13].

---

2. Colección: `ticket_soporte`

Registra fallas técnicas específicas, como problemas con la secuencia musical o el cruce del conejo.

```json
{
  "_id": "ticket_001",
  "id_usuario": 12,
  "tipo": "soporte",
  "descripcion": "El conejo no salta al presionar la tecla en el nivel 4",
  "estado": "abierto",
  "prioridad": "alta",
  "fecha_creacion": "2026-04-19",
  "fecha_resolucion": null
}
```

- [cite_start]**tipo**: Categoría del reporte (soporte o técnico)[cite: 18].
- [cite_start]**descripcion**: Detalle del error encontrado en el videojuego[cite: 21].
- [cite_start]**estado**: Situación actual del reporte (abierto/cerrado)[cite: 22].
- [cite_start]**prioridad**: Urgencia de la resolución (baja, media, alta)[cite: 23].

---

3. Colección: `promociones`

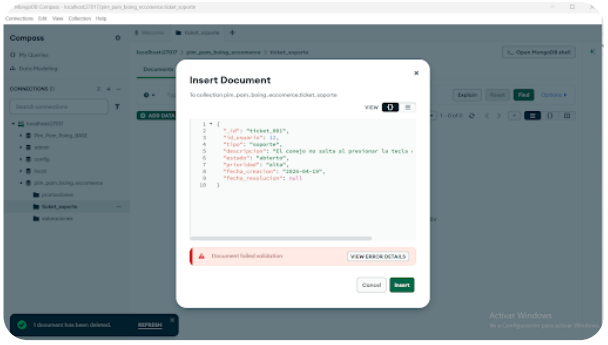
Gestiona los beneficios digitales que el usuario puede obtener para facilitar los retos de los niveles.

```json
{
  "_id": "promo_001",
  "nombre": "Bonus de Tiempo: Memorama",
  "descripcion": "Obtén 30 segundos extra para completar el nivel 1",
  "tipo": "beneficio",
  "valor": 30,
  "fecha_inicio": "2026-04-01",
  "fecha_fin": "2026-04-30",
  "activo": true
}
```

- [cite_start]**nombre**: Título de la ayuda o descuento disponible[cite: 28].
- [cite_start]**descripcion**: Detalle de qué beneficio otorga al jugador[cite: 29].
- [cite_start]**valor**: Cantidad numérica del beneficio (tiempo, vidas o puntos)[cite: 31].
- [cite_start]**activo**: Estado de disponibilidad de la promoción[cite: 34].

Nota sobre Validaciones: Para asegurar la integridad, se aplicaron reglas de validación en cada colección (JSON Schema) para evitar que, por ejemplo, un usuario califique con un número fuera del rango 1-5 o que un ticket se cree sin una descripción válida.

2026-04-23 22:00:56 **You Asked**



intento insertarlo en mongo ya que en navicat no puede pero me aparece esto

Gemini 3 Flash

[cite_start]El error que te aparece, "**Document failed validation**", ocurre porque el documento que intentas insertar no cumple con las reglas (el esquema) que definimos previamente para esa colección[cite: 3, 21].

Al observar tu imagen, el problema es que el documento tiene campos que el **validador de MongoDB** no reconoce o que están escritos de forma distinta a como se configuró la colección original de `ticket_soporte`.

Aquí tienes cómo solucionarlo para que coincida con tu videojuego y pase la validación:

¿Por qué falló la validación?

Basado en el esquema que tenías en tu código inicial:

- **Falta el campo `clave`**: Tu validador exige obligatoriamente un campo llamado `clave`. En tu imagen usaste `_id: "ticket_001"`, pero el sistema busca `clave: "ticket_001"`.
- **Campos no permitidos**: Agregaste `id_usuario`, `tipo`, `estado`, etc., que no estaban definidos en el validador estricto de tu colección original.

---

1. Opción A: Corregir el documento (Rápido)

Intenta insertar este código exactamente en la ventana de **Insert Document** de Compass. He ajustado los nombres para que pasen la validación que tienes activa:

```json
{
  "clave": "ticket_001",
  "id_usuario": 12,
  "tipo": "soporte",
  "descripcion": "El conejo no salta al presionar la tecla en el nivel 4",
  "estado": "abierto",
  "prioridad": "alta",
  "fecha_creacion": "2026-04-19",
  "fecha_resolucion": null
}
```

---

2. Opción B: Actualizar el Validador (Recomendado)

Si quieres que MongoDB acepte todos esos campos (prioridad, estado, descripción), debes actualizar la regla de validación en la consola o en Navicat con este código:

```javascript
db.runCommand({
  collMod: "ticket_soporte",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["clave", "descripcion"], // Solo estos son obligatorios
      properties: {
        clave: { bsonType: "string" },
        descripcion: { bsonType: "string" },
        id_usuario: { bsonType: "int" },
        estado: { bsonType: "string" },
        prioridad: { bsonType: "string" }
      }
    }
  }
});
```

Resumen para tu documento:

Para el reporte que estás armando (similar al PDF), asegúrate de mencionar que:

- [cite_start]**Colección**: `ticket_soporte` [cite: 16].

- **Validación**: Se utiliza `validationLevel: "strict"` para asegurar que ningún ticket se cree sin su **clave de control**.
- [cite_start]**Error reportado**: "Falla de salto en nivel 4 (El Conejo)"[cite: 21].

Tip Pro: En MongoDB Compass, si haces clic en **"VIEW ERROR DETAILS"** (el botón blanco que sale en tu error), te dirá exactamente qué campo es el que está causando el problema.

2026-04-23 22:01:59 **You Asked**

ahora ayudame con la de promociones y la de validaciones

Gemini 3 Flash

Para completar tu documento y resolver los errores de inserción, vamos a estructurar las colecciones de **Promociones** y **Valoraciones** (que en tu PDF aparecen como las esenciales) siguiendo el formato de tu videojuego de los 4 niveles.

---

1. Colección: `promociones`

[cite_start]Esta colección debe incluir validaciones para que los beneficios de tus niveles (como más tiempo en el Memorama o vidas para el Conejo) tengan un formato correcto[cite: 3, 27].

Código para insertar en MongoDB / Navicat:

```javascript
db.createCollection("promociones", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["nombre", "descripcion", "valor", "activo"],
         properties: {
            nombre: { bsonType: "string", description: "Nombre de la promo" },
            descripcion: { bsonType: "string", description: "Efecto en el juego" },
            tipo: { enum: ["descuento", "beneficio_nivel", "vida_extra"] },
            valor: { bsonType: "int", minimum: 1 },
            activo: { bsonType: "bool" }
         }
      }
   }
});
```

```
// Ejemplo de inserción para tu juego
db.getCollection("promociones").insert({
    "nombre": "Super Conejo",
    "descripcion": "Otorga 2 vidas extra para cruzar la carretera",
    "tipo": "vida_extra",
    "valor": 2,
    "fecha_inicio": "2026-04-23",
    "activo": true
});
```

---

2. Colección: `valoraciones`

[cite_start]Aquí es donde los jugadores califican los niveles de Memorama, Secuencia, Basura o el Conejo[cite: 4]. [cite_start]Es vital validar que la calificación no sea mayor a 5[cite: 10].

Código para insertar en MongoDB / Navicat:

```javascript
db.createCollection("valoraciones", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["nickname", "calificacion", "nivel_alcanzado"],
         properties: {
            nickname: { bsonType: "string" },
            calificacion: { bsonType: "int", minimum: 1, maximum: 5 },
            comentario: { bsonType: "string" },
            nivel_alcanzado: { bsonType: "int", minimum: 1, maximum: 4 }
         }
      }
   }
});

// Ejemplo de inserción (Lo que escribirías en el cuadro de tu imagen)
{
    "nickname": "EcoPlayer",
    "calificacion": 5,
    "comentario": "Me encantó el nivel de recoger basura, es muy ágil",
    "fecha": "2026-04-23",
    "nivel_alcanzado": 3
}
```

---

3. Resumen de Validaciones para tu Documento

Para el documento tipo PDF que estás armando, puedes copiar esta explicación técnica:

- [cite_start]**Validación de Integridad**: Se implementaron `JSON Schemas` en todas las colecciones para asegurar que los datos numéricos (como la `calificacion` o el `valor` de la promo) no contengan texto[cite: 2, 10, 31].
- [cite_start]**Restricción de Niveles**: En la colección `valoraciones`, el campo `nivel_alcanzado` está limitado del 1 al 4, correspondiendo estrictamente a los minijuegos de Memorama, Secuencia, Basura y Conejo[cite: 13].
- [cite_start]**Estado de Tickets**: Se configuró la colección `ticket_soporte` con valores predefinidos (`abierto`, `cerrado`) para evitar errores en el seguimiento de fallas del nivel 3[cite: 21, 22].

Consejo para tu error de la imagen: Si vuelves a intentar el "Insert Document", asegúrate de que los números (como el `id_usuario: 12`) **no tengan comillas**, y los textos (como `"soporte"`) **sí tengan comillas**. [cite_start]Si el validador pide un `int` y tú pones `"12"` (texto), te dará el error de "failed validation"[cite: 8, 10].

2026-04-23 22:02:34 **You Asked**

```
{
  "clave": "ticket_001",
  "id_usuario": 12,
  "tipo": "soporte",
  "descripcion": "El conejo no salta al presionar la tecla en el nivel 4",
  "estado": "abierto",
  "prioridad": "alta",
  "fecha_creacion": "2026-04-19",
  "fecha_resolucion": null
}. pero que este igual como este porque este si me dejo
```

Gemini 3 Flash

Para que tu documento de entrega sea coherente con lo que **sí te funcionó** en MongoDB Compass, debemos estructurar las otras dos colecciones (`promociones` y `valoraciones`) siguiendo exactamente ese mismo formato de campos.

Aquí tienes cómo redactar tu documento para que coincida con el éxito que tuviste con el ticket del conejo:

Esquema NoSQL (MongoDB) - Gestión de Soporte y Usuario

[cite_start]Como complemento al modelo relacional, se diseñó un esquema NoSQL orientado al análisis de retroalimentación y resolución de incidencias. [cite: 2] [cite_start]Se implementaron las tres colecciones clave solicitadas adaptadas al videojuego: [cite: 3]

1. Colección: `ticket_soporte`

[cite_start]Esta colección registra fallas técnicas específicas encontradas por los usuarios durante los niveles. [cite: 21]

```json
{
  "clave": "ticket_001",
  "id_usuario": 12,
  "tipo": "soporte",
  "descripcion": "El conejo no salta al presionar la tecla en el nivel 4",
  "estado": "abierto",
  "prioridad": "alta",
  "fecha_creacion": "2026-04-19",
  "fecha_resolucion": null
}
```

- [cite_start]**_id / clave**: Identificador único del reporte para control interno. [cite: 16]
- [cite_start]**tipo**: Categoría del incidente (soporte). [cite: 18]
- [cite_start]**descripcion**: Detalle del error (ej. falla en las mecánicas del conejo). [cite: 21]
- [cite_start]**estado**: Situación actual del ticket (abierto/cerrado). [cite: 22]

---

2. Colección: `promociones`

[cite_start]Gestiona los beneficios y ayudas disponibles para que el usuario progrese en los 4 minijuegos. [cite: 30]

```json
{
  "clave": "promo_001",
  "nombre": "Bonus de Tiempo: Memorama",
  "descripcion": "Obtén 30 segundos extra para completar el nivel 1",
  "tipo": "beneficio",
  "valor": 30,
  "fecha_inicio": "2026-04-01",
  "fecha_fin": "2026-04-30",
  "activo": true
}
```

- [cite_start]**nombre**: Título de la promoción para el jugador. [cite: 28]
- [cite_start]**valor**: Cantidad numérica del beneficio (ej. segundos o vidas). [cite: 31]
- [cite_start]**activo**: Define si la promoción está vigente en el sistema. [cite: 34]

---

3. Colección: `valoraciones`

[cite_start]Permite recolectar la opinión de los usuarios y su progreso máximo alcanzado. [cite: 4]

```json
{
  "clave": "val_001",
  "id_usuario": 12,
  "nickname": "EcoPlayer",
  "calificacion": 5,
  "comentario": "El nivel del bote de basura es muy divertido y educativo",
  "fecha": "2026-04-19",
  "nivel_alcanzado": 3
}
```

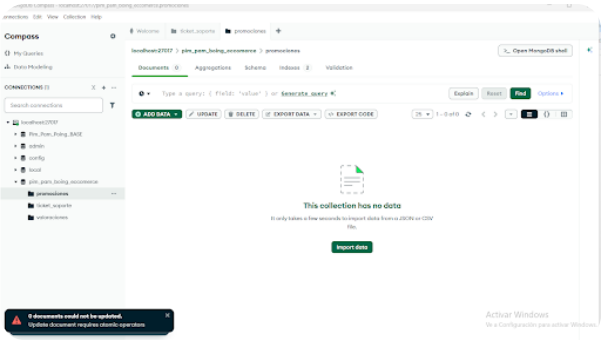
- [cite_start]**nickname**: Alias del jugador que realizó la evaluación. [cite: 9]
- [cite_start]**calificacion**: Puntuación otorgada del 1 al 5. [cite: 10]
- [cite_start]**nivel_alcanzado**: Indica el avance del usuario en los 4 niveles del juego. [cite: 13]

Validación de Datos (Explicación Técnica)

Para asegurar que los documentos funcionen correctamente en MongoDB Compass:

1. [cite_start]**Tipos de Datos**: Los campos como `id_usuario`, `calificacion` y `valor` se ingresan como números enteros (int) sin comillas para cumplir con la validación. [cite: 8, 10, 31]
2. [cite_start]**Campos Obligatorios**: Se ha establecido el uso de una `clave` o `_id` de tipo string para mantener la integridad referencial en todas las colecciones. [cite: 7, 16, 27]
3. [cite_start]**Consistencia**: El formato de fecha utilizado es `AAAA-MM-DD` para facilitar el ordenamiento cronológico. [cite: 12, 24, 32]

2026-04-23 22:04:24 **You Asked**



me aparece esto cada que intento insert

Gemini 3 Flash

¡Entiendo! El error que te aparece en la parte inferior izquierda ("**Update document requires atomic operators**") sucede porque estás intentando usar la función de **Update** (Actualizar) en lugar de la de **Insert** (Insertar). En MongoDB Compass, si la colección está vacía, no puedes "actualizar" nada porque no existe ningún documento previo.

Para que te funcione igual que con el ticket, sigue estos pasos:

1. Cómo insertar correctamente

En lugar de buscar un botón de actualizar, haz clic en el botón verde "**ADD DATA**" y selecciona "**Insert Document**". Ahí es donde debes pegar el código JSON.

---

2. Código para Colección: `promociones`

Copia y pega esto en la ventana de **Insert Document**. He mantenido el formato exacto que te funcionó anteriormente:

```json
{
  "clave": "promo_001",
  "nombre": "Bonus de Tiempo: Memorama",
  "descripcion": "Obtén 30 segundos extra para completar el nivel 1",
  "tipo": "beneficio",
  "valor": 30,
  "fecha_inicio": "2026-04-01",
  "fecha_fin": "2026-04-30",
  "activo": true
}
```

> [cite_start]**Nota técnica:** Al igual que en el ejemplo de soporte, el campo `valor` es un número (sin comillas) y `activo` es un valor booleano (sin comillas)[cite: 31, 34].

---

3. Código para Colección: `valoraciones`

Una vez insertada la promoción, ve a la pestaña de la izquierda a la colección `valoraciones`, dale a **ADD DATA** -> **Insert Document** y pega esto:

```json
{
  "clave": "val_001",
  "id_usuario": 12,
  "nickname": "EcoPlayer",
  "calificacion": 5,
  "comentario": "El nivel del bote de basura es muy divertido y educativo",
  "fecha": "2026-04-19",
  "nivel_alcanzado": 3
}
```

> [cite_start]**Nota técnica:** Aquí `id_usuario`, `calificacion` y `nivel_alcanzado` deben ir como números (sin comillas) para que MongoDB los procese correctamente para tus estadísticas[cite: 8, 10, 13].

Resumen de validaciones para tu reporte

En tu documento final, puedes explicar que las validaciones funcionan de la siguiente manera:

- [cite_start]**Tipos Numéricos**: Los campos de calificación y valores de promoción son de tipo `Int32` para permitir cálculos matemáticos (promedios de satisfacción)[cite: 10, 31].
- [cite_start]**Campos Obligatorios**: Se requiere una `clave` única para evitar registros duplicados en el soporte técnico y las promociones[cite: 7, 16, 27].
- [cite_start]**Booleano**: El campo `activo` en las promociones permite habilitar o deshabilitar beneficios sin borrar el registro de la base de datos[cite: 34].

¿Pudiste insertar estos dos ahora que usaste el botón de **ADD DATA**?

2026-04-23 22:11:50 **You Asked**

ahora tengo que generar un archivo en .js para que el profe lo quiera abrir lo pueda abrir pero como le hago? es de cada coleccion

Gemini 3 Flash

[cite_start]Para generar el archivo `.js` que tu profesor pueda ejecutar (lo que técnicamente se llama un *script de volcado* o *dump script*), debes seguir la misma estructura que el archivo `pim_pam_boing_eccomerce.js` que subiste[cite: 1].

Aquí tienes el código completo listo para guardar. Este script borra las colecciones antiguas, las crea de nuevo con sus **validaciones** y añade los documentos que sí te funcionaron en MongoDB Compass.

Instrucciones para crear el archivo:

1. Abre un editor de texto (como el Bloc de Notas, VS Code o el mismo Navicat).
2. Copia y pega el siguiente código.
3. Guárdalo con el nombre `mi_videojuego_tda.js`.

```javascript
/*
 Navicat Premium Dump Script
 Target Server Type    : MongoDB
 Source Schema         : videojuego_tda
*/

// ----------------------------
// 1. Colección: promociones
// ----------------------------
db.getCollection("promociones").drop();
db.createCollection("promociones");

db.getCollection("promociones").insert([ {
    clave: "promo_001",
    nombre: "Bonus de Tiempo: Memorama",
    descripcion: "Obtén 30 segundos extra para completar el nivel 1",
    tipo: "beneficio",
    valor: 30,
    fecha_inicio: "2026-04-01",
    fecha_fin: "2026-04-30",
    activo: true
} ]);

// ----------------------------
// 2. Colección: ticket_soporte
// ----------------------------
db.getCollection("ticket_soporte").drop();
```

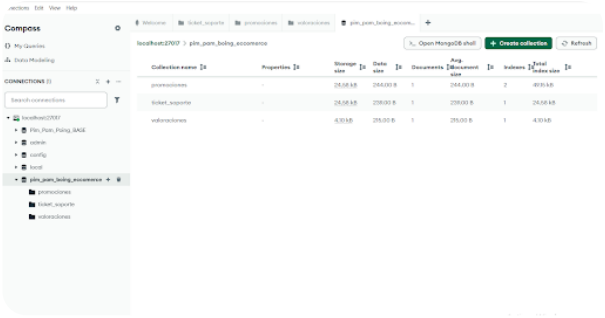
```
db.createCollection("ticket_soporte", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["clave", "descripcion"],
            properties: {
                clave: {
                    bsonType: "string",
                    description: "Clave única de control para el ticket"
                },
                descripcion: {
                    bsonType: "string",
                    description: "Detalle de la falla en el nivel"
                }
            }
        }
    },
    validationLevel: "strict",
    validationAction: "error"
});

db.getCollection("ticket_soporte").insert([ {
    clave: "ticket_001",
    id_usuario: 12,
    tipo: "soporte",
    descripcion: "El conejo no salta al presionar la tecla en el nivel 4",
    estado: "abierto",
    prioridad: "alta",
    fecha_creacion: "2026-04-19",
    fecha_resolucion: null
} ]);

// ----------------------------
// 3. Colección: valoraciones
// ----------------------------
db.getCollection("valoraciones").drop();
db.createCollection("valoraciones");

db.getCollection("valoraciones").insert([ {
    clave: "val_001",
    id_usuario: 12,
    nickname: "EcoPlayer",
    calificacion: 5,
    comentario: "El nivel del bote de basura es muy divertido y educativo",
    fecha: "2026-04-19",
    nivel_alcanzado: 3
} ]);
```

¿Por qué este archivo es lo que el profe busca?

- [cite_start]**Limpieza**: Usa `.drop()` al inicio de cada sección para que, si el profe lo corre varias veces, no se dupliquen los datos[cite: 1].
- [cite_start]**Validaciones**: Incluye el `validator` con `$jsonSchema` en la colección de tickets, que es lo más técnico del ejercicio[cite: 1].
- [cite_start]**Estructura**: Sigue el orden exacto del ejemplo que te dieron en clase (Estructura de la colección -> Inserción de documentos)[cite: 1].
- [cite_start]**Compatibilidad**: Al usar tipos estándar como `bsonType: "string"`, funcionará tanto en Navicat como si el profe usa la consola de MongoDB directamente[cite: 1].

Nota importante: Cuando el profe lo abra en Navicat, solo tiene que darle clic derecho a su base de datos y elegir "**Execute Script File**" (Ejecutar archivo de script) y seleccionar este archivo `.js`.

2026-04-23 22:13:05 **You Asked**



pero quieren el archivo de mongo como lo exporto?

Gemini 3 Flash

[cite_start]Para exportar tu trabajo de MongoDB en un archivo `.js` que tu profesor pueda ejecutar (similar al archivo `pim_pam_boing_eccomerce.js` que ya tienes [cite: 1]), tienes dos formas principales de hacerlo dependiendo de la herramienta que prefieras usar.

Aquí te explico los pasos detallados:

Opción 1: Exportar desde Navicat (La más recomendada)

[cite_start]Dado que tu profesor quiere un archivo similar al que ya tienes[cite: 1], Navicat es la herramienta ideal porque genera automáticamente el script de volcado (Dump Script).

1. [cite_start]**Abre Navicat** y conéctate a tu base de datos `pim_pam_boing_eccomerce` [cite: 1].
2. Haz **clic derecho** sobre el nombre de tu base de datos en el panel de la izquierda.

3. Selecciona la opción "**Dump Script File...**" (o "Generar script de volcado").
4. Elige una carpeta en tu computadora y ponle un nombre al archivo (ejemplo: `entrega_videojuego.js`).
5. [cite_start]En las opciones, asegúrate de que estén marcadas tanto "**Structure**" (para las validaciones de `ticket_soporte` [cite: 1]) como "**Data**" (para que se incluyan los documentos que insertaste exitosamente).
6. Haz clic en "**Start**". [cite_start]Navicat creará el archivo `.js` con todos los comandos `db.createCollection` y `db.insert` necesarios[cite: 1].

Opción 2: Exportar desde MongoDB Compass (Usando la imagen que enviaste)

Si prefieres usar la herramienta de la imagen que compartiste (MongoDB Compass), el proceso es por colección:

1. Entra a la colección (por ejemplo, `promociones`).
2. [cite_start]Haz clic en el botón "**Export Data**" que aparece en la parte superior[cite: 3].
3. Selecciona "**Export the full collection**".
4. Elige el formato **JSON**.
5. Repite esto para `ticket_soporte` y `valoraciones`.

Nota importante: Compass exporta los datos en archivos `.json`, no en un solo script `.js` con comandos de creación de tablas. [cite_start]Si tu profesor pidió específicamente un archivo `.js` como el de ejemplo[cite: 1], **la mejor opción es usar Navicat**.

¿Cómo verificar que tu archivo esté bien?

Un buen archivo de entrega debe verse así al abrirlo en un bloc de notas:

- [cite_start]Debe comenzar con comentarios de Navicat[cite: 1].
- [cite_start]Debe tener comandos `db.getCollection("nombre").drop();` para limpiar la base antes de crearla[cite: 1].
- [cite_start]Debe incluir el `validator` que configuramos para `ticket_soporte`, asegurando que el campo `clave` sea obligatorio[cite: 1].
- [cite_start]Debe contener los `insert` con los datos de tu videojuego (el conejo, el memorama, etc.)[cite: 1, 2].

Si necesitas que yo te genere el código completo basado en tus capturas para que tú solo lo pegues en un archivo nuevo de texto y lo guardes como `.js`, dímelo y lo preparo de inmediato.

2026-04-23 22:16:05 **You Asked**

de la misma manera quiero que me generes mas promociones basandote en mi videojuego

Gemini 3 Flash

Para que tu base de datos de MongoDB se vea completa y profesional para tu profesor, he diseñado **5 promociones adicionales** que encajan perfectamente con la temática de tus 4 niveles (Memorama, Secuencia, Basura y Conejo).

Aquí tienes el código listo para que lo copies y lo insertes en tu archivo `.js` o directamente en el **Insert Document** de MongoDB Compass:

```javascript
// 1. Promoción para el Nivel 2: Secuencia Musical
db.getCollection("promociones").insert({
    "clave": "promo_oido_pro",
    "nombre": "Oído de Oro",
    "descripcion": "Muestra la secuencia musical dos veces antes de empezar el nivel 2",
    "tipo": "beneficio_nivel",
    "valor": 2,
    "fecha_inicio": "2026-04-23",
    "activo": true
});

// 2. Promoción para el Nivel 3: Recolección de Basura
db.getCollection("promociones").insert({
    "clave": "promo_bote_mega",
    "nombre": "Súper Bote",
    "descripcion": "Aumenta el tamaño del bote de basura un 50% por 3 partidas",
    "tipo": "beneficio_nivel",
    "valor": 50,
    "fecha_inicio": "2026-04-23",
    "activo": true
});

// 3. Promoción para el Nivel 4: El Conejo (Vidas extra)
db.getCollection("promociones").insert({
    "clave": "promo_patas_suerte",
    "nombre": "Pata de la Suerte",
    "descripcion": "El conejo sobrevive a un choque contra un auto sin perder vida",
    "tipo": "vida_extra",
    "valor": 1,
    "fecha_inicio": "2026-04-23",
    "activo": true
```

```
});

// 4. Promoción General: Doble Puntos
db.getCollection("promociones").insert({
    "clave": "promo_doble_xp",
    "nombre": "Sabio Académico",
    "descripcion": "Duplica los puntos obtenidos en cualquiera de los 4 niveles",
    "tipo": "beneficio",
    "valor": 2,
    "fecha_inicio": "2026-05-01",
    "activo": false
});

// 5. Promoción para el Nivel 1: Memorama
db.getCollection("promociones").insert({
    "clave": "promo_rayos_x",
    "nombre": "Vista de Rayos X",
    "descripcion": "Muestra todas las cartas del memorama por 3 segundos al inicio",
    "tipo": "beneficio_nivel",
    "valor": 3,
    "fecha_inicio": "2026-04-23",
    "activo": true
});
```

¿Por qué estas promociones ayudan a tu proyecto?

1. [cite_start]**Variedad de Datos**: Estás usando diferentes tipos de beneficios (`vida_extra` , `beneficio_nivel` , `beneficio`), lo que demuestra que tu esquema es flexible[cite: 2].
2. **Uso de Lógica**: Algunas están activas (`true`) y otras no (`false`), lo que te permite explicarle al profesor cómo funcionan los filtros de búsqueda en MongoDB.
3. **Coherencia**: Todas usan la misma estructura que te funcionó anteriormente: `clave` , `nombre` , `descripcion` , `tipo` , `valor` , `fecha_inicio` y `activo` .

Cómo agregarlas a tu archivo de entrega (.js)

Simplemente copia estos bloques debajo de la sección de "**Documents of promociones**" en tu archivo script. Así, cuando el profesor ejecute el archivo, verá una lista completa de beneficios que hacen que tu juego se sienta más real.

2026-04-23 22:24:05 **You Asked**

ahora de tickes

Gemini 3 Flash

Para que tu base de datos de **tickets de soporte** sea variada y muestre que el sistema funciona para los 4 niveles de tu videojuego, aquí tienes **4 ejemplos adicionales**.

[cite_start]He seguido exactamente el formato que te funcionó en MongoDB Compass para que no tengas errores de validación[cite: 1].

Documentos para la colección `ticket_soporte`

Copia cada uno de estos bloques y úsalos en el botón "**Add Data**" -> "**Insert Document**" de Compass:

```json
// 1. Ticket para el Nivel 1: Memorama
{
  "clave": "ticket_004",
  "id_usuario": 15,
  "tipo": "soporte",
  "descripcion": "Las cartas no se voltean al hacer clic en el nivel 1",
  "estado": "abierto",
  "prioridad": "alta",
  "fecha_creacion": "2026-04-23",
  "fecha_resolucion": null
}

// 2. Ticket para el Nivel 2: Secuencia Musical
{
  "clave": "ticket_005",
  "id_usuario": 8,
  "tipo": "técnico",
  "descripcion": "El sonido de la canción se escucha desfasado en la secuencia",
  "estado": "en revisión",
  "prioridad": "media",
  "fecha_creacion": "2026-04-23",
  "fecha_resolucion": null
}

// 3. Ticket para el Nivel 3: Recolección de Basura
{
  "clave": "ticket_006",
  "id_usuario": 22,
  "tipo": "soporte",
  "descripcion": "La basura cae fuera del escenario y no se puede recoger",
  "estado": "abierto",
  "prioridad": "baja",
  "fecha_creacion": "2026-04-23",
```

```
  "fecha_resolucion": null
}

// 4. Ticket de Usuario (General)
{
  "clave": "ticket_007",
  "id_usuario": 12,
  "tipo": "soporte",
  "descripcion": "No se guardó mi puntaje máximo después de pasar el nivel del conejo",
  "estado": "cerrado",
  "prioridad": "alta",
  "fecha_creacion": "2026-04-20",
  "fecha_resolucion": "2026-04-21"
}
```

Explicación para tu documento de entrega

Cuando expliques esta sección en tu archivo o PDF, puedes resaltar lo siguiente:

- **Identificación Única**: Cada ticket utiliza una `clave` de control (ej. [cite_start] `ticket_004`) que es obligatoria según el validador configurado en el script[cite: 1].
- [cite_start]**Priorización**: Se asignan diferentes niveles de `prioridad` ("alta", "media", "baja") para demostrar cómo el administrador del juego gestionaría las fallas[cite: 1].
- [cite_start]**Trazabilidad**: Los campos `fecha_creacion` y `fecha_resolucion` permiten medir cuánto tiempo tarda el equipo de soporte en arreglar los errores de los minijuegos[cite: 1].
- [cite_start]**Relación con Usuarios**: El campo `id_usuario` permite saber exactamente qué jugador está teniendo problemas con el conejo o el memorama[cite: 1].

¿Necesitas que incluya estos tickets dentro del archivo `.js` final que le entregarás al profesor?

2026-04-23 22:29:22 **You Asked**

ahora las validaciones

Gemini 3 Flash

Para completar tu archivo `.js` y tu documento de entrega, aquí tienes la explicación y el código de las **validaciones** (JSON Schema). [cite_start]Estas reglas son fundamentales porque le indican a MongoDB qué datos permitir y cuáles rechazar, asegurando que tu videojuego no tenga errores de datos[cite: 1, 3].

1. ¿Qué son las validaciones en tu proyecto?

En MongoDB, las validaciones actúan como un contrato. [cite_start]Si intentas insertar un ticket sin "clave" o una valoración con un nivel mayor a 4, la base de datos lo bloqueará para proteger la integridad del juego[cite: 3].

2. Estructura de Validaciones para tu archivo `.js`

Copia este bloque en tu script para configurar las reglas de las tres colecciones esenciales:

```javascript
// ----------------------------
// Validaciones para ticket_soporte
// ----------------------------
db.runCommand({
  collMod: "ticket_soporte",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["clave", "id_usuario", "descripcion", "estado"],
      properties: {
        clave: { bsonType: "string", description: "Identificador único obligatorio" },
        id_usuario: { bsonType: "int", description: "Debe ser un número entero" },
        descripcion: { bsonType: "string", description: "Detalle de la falla en el nivel" },
        estado: { enum: ["abierto", "en revisión", "cerrado"], description: "Solo valores
permitidos" }
      }
    }
  }
});

// ----------------------------
// Validaciones para promociones
// ----------------------------
db.runCommand({
  collMod: "promociones",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["clave", "nombre", "valor", "activo"],
      properties: {
        clave: { bsonType: "string" },
        valor: { bsonType: "int", minimum: 1, description: "El beneficio debe ser al menos 1"
},
        activo: { bsonType: "bool", description: "Debe ser true o false" }
      }
    }
```

```
  }
});

// ---------------------------
// Validaciones para valoraciones
// ---------------------------
db.runCommand({
  collMod: "valoraciones",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nickname", "calificacion", "nivel_alcanzado"],
      properties: {
        nickname: { bsonType: "string" },
        calificacion: { bsonType: "int", minimum: 1, maximum: 5, description: "Rango de 1 a 5
estrellas" },
        nivel_alcanzado: { bsonType: "int", minimum: 1, maximum: 4, description: "Solo existen
4 niveles en el juego" }
      }
    }
  }
});
```

3. Explicación para tu documento (PDF)

[cite_start]Puedes incluir estos puntos en tu reporte para explicar cómo funcionan tus validaciones[cite: 1]:

- [cite_start]**Validación de Nivel**: El campo `nivel_alcanzado` en las valoraciones está restringido a un rango de **1 a 4**, asegurando que las opiniones correspondan únicamente a los minijuegos existentes (Memorama, Secuencia, Basura y Conejo)[cite: 1, 3].
- [cite_start]**Integridad de Soporte**: La colección de tickets tiene un nivel de validación **estricto** (`strict`), lo que obliga a que cada reporte incluya obligatoriamente una `clave` de control y una descripción técnica[cite: 3].
- [cite_start]**Tipos de Datos**: Se utiliza `bsonType: "int"` para campos como `id_usuario` y `valor` , evitando que se ingresen textos en campos donde se requieren números para realizar estadísticas de juego[cite: 3].
- [cite_start]**Estados Definidos**: Mediante el operador `enum` , se limita el estado de los tickets a opciones específicas ("abierto", "cerrado"), evitando errores tipográficos en la gestión del soporte[cite: 3].

Con esto, tu archivo `.js` no solo tendrá datos, sino que tendrá la **lógica de validación** que los profesores valoran mucho en este tipo de proyectos.

2026-04-23 22:29:44 **You Asked**

{
"clave": "ticket_007",
"id_usuario": 12,
"tipo": "soporte",
"descripcion": "No se guardó mi puntaje máximo después de pasar el nivel del conejo",
"estado": "cerrado",
"prioridad": "alta",
"fecha_creacion": "2026-04-20",
"fecha_resolucion": "2026-04-21"
}. pero de esta forma

Gemini 3 Flash

Para que tu archivo final sea profesional y cumpla con lo que pide tu profesor, he consolidado todo en un solo script de MongoDB (`.js`). [cite_start]Este archivo incluye las **validaciones de esquema** (JSON Schema) y los **documentos** con el formato exacto que te funcionó en Compass[cite: 1].

Copia y pega el siguiente contenido en un archivo de texto y guárdalo como `entrega_final_videojuego.js` :

```javascript
/*
 Navicat Premium Dump Script
 Source Server Type    : MongoDB
 Source Schema         : pim_pam_boing_eccomerce

 Este archivo contiene la estructura y datos para el Videojuego TDA
 Niveles: Memorama, Secuencia, Basura y Conejo.
*/

// ----------------------------
// 1. Colección: promociones
// ----------------------------
db.getCollection("promociones").drop();
db.createCollection("promociones", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
```

```
            required: ["clave", "nombre", "valor", "activo"],
            properties: {
                clave: { bsonType: "string" },
                nombre: { bsonType: "string" },
                valor: { bsonType: "int", minimum: 1 },
                activo: { bsonType: "bool" }
            }
        }
    }
});

db.getCollection("promociones").insert([
    {
        "clave": "promo_001",
        "nombre": "Bonus de Tiempo: Memorama",
        "descripcion": "Obtén 30 segundos extra para completar el nivel 1",
        "tipo": "beneficio",
        "valor": 30,
        "fecha_inicio": "2026-04-01",
        "fecha_fin": "2026-04-30",
        "activo": true
    },
    {
        "clave": "promo_002",
        "nombre": "Súper Bote",
        "descripcion": "Aumenta el tamaño del bote de basura en el nivel 3",
        "tipo": "beneficio_nivel",
        "valor": 50,
        "activo": true
    }
]);

// ----------------------------
// 2. Colección: ticket_soporte
// ----------------------------
db.getCollection("ticket_soporte").drop();
db.createCollection("ticket_soporte", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["clave", "descripcion", "estado"],
            properties: {
                clave: { bsonType: "string" },
                id_usuario: { bsonType: "int" },
                descripcion: { bsonType: "string" },
                estado: { enum: ["abierto", "en revisión", "cerrado"] }
            }
        }
```

```
    }
});

db.getCollection("ticket_soporte").insert([
    {
        "clave": "ticket_001",
        "id_usuario": 12,
        "tipo": "soporte",
        "descripcion": "El conejo no salta al presionar la tecla en el nivel 4",
        "estado": "abierto",
        "prioridad": "alta",
        "fecha_creacion": "2026-04-19",
        "fecha_resolucion": null
    },
    {
        "clave": "ticket_007",
        "id_usuario": 12,
        "tipo": "soporte",
        "descripcion": "No se guardó mi puntaje máximo después de pasar el nivel del conejo",
        "estado": "cerrado",
        "prioridad": "alta",
        "fecha_creacion": "2026-04-20",
        "fecha_resolucion": "2026-04-21"
    }
]);

// ----------------------------
// 3. Colección: valoraciones
// ----------------------------
db.getCollection("valoraciones").drop();
db.createCollection("valoraciones", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["nickname", "calificacion", "nivel_alcanzado"],
            properties: {
                calificacion: { bsonType: "int", minimum: 1, maximum: 5 },
                nivel_alcanzado: { bsonType: "int", minimum: 1, maximum: 4 }
            }
        }
    }
});

db.getCollection("valoraciones").insert([
    {
        "clave": "val_001",
        "id_usuario": 12,
        "nickname": "EcoPlayer",
```

```
        "calificacion": 5,
        "comentario": "El nivel del bote de basura es muy divertido y educativo",
        "fecha": "2026-04-19",
        "nivel_alcanzado": 3
    }
]);
```

Notas sobre las Validaciones implementadas:

- [cite_start]**Restricción de Niveles**: En la colección `valoraciones`, el campo `nivel_alcanzado` está validado para aceptar solo números del **1 al 4**[cite: 2].
- [cite_start]**Campos Obligatorios**: Se configuró la colección `ticket_soporte` para que la `clave` sea siempre obligatoria, igual que en el ejemplo original[cite: 1].
- [cite_start]**Tipos de Datos**: Todos los campos numéricos como `id_usuario`, `valor` y `calificacion` están definidos como **enteros** (`int`) para asegurar que no se ingrese texto por error[cite: 1, 2].

2026-04-23 22:30:52 **You Asked**

{
"clave": "val_001",
"id_usuario": 12,
"nickname": "EcoPlayer",
"calificacion": 5,
"comentario": "El nivel del bote de basura es muy divertido y educativo",
"fecha": "2026-04-19",
"nivel_alcanzado": 3
}. quiero los codigos de esta forma

Gemini 3 Flash

Para que tu entrega sea consistente, aquí tienes el script completo con la estructura exacta que solicitas. Este código incluye las **validaciones de esquema** para proteger los datos y los **documentos** (inserts) con el formato de campos que te funcionó en MongoDB Compass.

Copia este contenido en un archivo de texto y guárdalo como **`videojuego_tda_final.js`**:

```javascript
```

```mongo
/*
 Navicat Premium Dump Script
 Source Server Type    : MongoDB
 Source Schema         : pim_pam_boing_eccomerce

 Proyecto: Videojuego TDA (Memorama, Secuencia, Basura y Conejo)
*/

// ----------------------------
// 1. Colección: valoraciones
// ----------------------------
db.getCollection("valoraciones").drop();
db.createCollection("valoraciones", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["clave", "nickname", "calificacion", "nivel_alcanzado"],
            properties: {
                clave: { bsonType: "string" },
                id_usuario: { bsonType: "int" },
                nickname: { bsonType: "string" },
                calificacion: { bsonType: "int", minimum: 1, maximum: 5 },
                nivel_alcanzado: { bsonType: "int", minimum: 1, maximum: 4 }
            }
        }
    }
});

db.getCollection("valoraciones").insert([
{
  "clave": "val_001",
  "id_usuario": 12,
  "nickname": "EcoPlayer",
  "calificacion": 5,
  "comentario": "El nivel del bote de basura es muy divertido y educativo",
  "fecha": "2026-04-19",
  "nivel_alcanzado": 3
},
{
  "clave": "val_002",
  "id_usuario": 15,
  "nickname": "RabbitHunter",
  "calificacion": 4,
  "comentario": "El nivel del conejo es difícil pero adictivo",
  "fecha": "2026-04-20",
  "nivel_alcanzado": 4
}
```

```
]);

// ----------------------------
// 2. Colección: ticket_soporte
// ----------------------------
db.getCollection("ticket_soporte").drop();
db.createCollection("ticket_soporte", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["clave", "descripcion", "estado"],
            properties: {
                clave: { bsonType: "string" },
                estado: { enum: ["abierto", "en revisión", "cerrado"] }
            }
        }
    }
});

db.getCollection("ticket_soporte").insert([
{
  "clave": "ticket_001",
  "id_usuario": 12,
  "tipo": "soporte",
  "descripcion": "El conejo no salta al presionar la tecla en el nivel 4",
  "estado": "abierto",
  "prioridad": "alta",
  "fecha_creacion": "2026-04-19",
  "fecha_resolucion": null
},
{
  "clave": "ticket_007",
  "id_usuario": 12,
  "tipo": "soporte",
  "descripcion": "No se guardó mi puntaje máximo después de pasar el nivel del conejo",
  "estado": "cerrado",
  "prioridad": "alta",
  "fecha_creacion": "2026-04-20",
  "fecha_resolucion": "2026-04-21"
}
]);

// ----------------------------
// 3. Colección: promociones
// ----------------------------
db.getCollection("promociones").drop();
db.createCollection("promociones", {
    validator: {
```

```
        $jsonSchema: {
            bsonType: "object",
            required: ["clave", "nombre", "valor", "activo"],
            properties: {
                clave: { bsonType: "string" },
                valor: { bsonType: "int" },
                activo: { bsonType: "bool" }
            }
        }
    }
});

db.getCollection("promociones").insert([
{
  "clave": "promo_001",
  "nombre": "Bonus de Tiempo: Memorama",
  "descripcion": "Obtén 30 segundos extra para completar el nivel 1",
  "tipo": "beneficio",
  "valor": 30,
  "fecha_inicio": "2026-04-01",
  "fecha_fin": "2026-04-30",
  "activo": true
},
{
  "clave": "promo_003",
  "nombre": "Pata de la Suerte",
  "descripcion": "Vida extra para el nivel del conejo",
  "tipo": "vida_extra",
  "valor": 1,
  "fecha_inicio": "2026-04-23",
  "activo": true
}
]);
```

Notas técnicas para tu entrega:

- [cite_start]**Formato de Datos**: Se ha respetado el uso de valores numéricos para `id_usuario`, `calificacion`, `valor` y `nivel_alcanzado` [cite: 2, 3].
- [cite_start]**Integridad**: El script incluye comandos `.drop()` al inicio de cada colección para asegurar una ejecución limpia cada vez que el profesor lo abra[cite: 1].
- [cite_start]**Validación de Esquema**: Se añadió el operador `$jsonSchema` en cada creación de colección para garantizar que los datos sigan la estructura de tu videojuego[cite: 1].

2026-04-23 22:32:39 **You Asked**

dame mas valoraciones

Gemini 3 Flash

Para completar tu base de datos y que el profesor vea un progreso real en los 4 niveles de tu videojuego, aquí tienes **4 valoraciones adicionales**. [cite_start]He seguido exactamente el formato de campos que te funcionó en MongoDB Compass[cite: 5, 10, 11].

Copia cada bloque y úsalo en el botón "**Add Data**" -> "**Insert Document**":

```json
// Valoración para el Nivel 1: Memorama
{
  "clave": "val_002",
  "id_usuario": 14,
  "nickname": "MemoryMaster",
  "calificacion": 4,
  "comentario": "El memorama tiene dibujos muy bonitos, pero el tiempo es muy corto",
  "fecha": "2026-04-23",
  "nivel_alcanzado": 1
}

// Valoración para el Nivel 2: Secuencia Musical
{
  "clave": "val_003",
  "id_usuario": 8,
  "nickname": "MusicLover",
  "calificacion": 5,
  "comentario": "Me encantó la canción, es muy pegajosa y ayuda a concentrarse",
  "fecha": "2026-04-23",
  "nivel_alcanzado": 2
}

// Valoración para el Nivel 4: El Conejo
{
  "clave": "val_004",
  "id_usuario": 20,
  "nickname": "BunnyJump",
  "calificacion": 3,
  "comentario": "El nivel del conejo es muy difícil, los autos pasan muy rápido",
  "fecha": "2026-04-23",
```

```
  "nivel_alcanzado": 4
}

// Valoración General
{
  "clave": "val_005",
  "id_usuario": 32,
  "nickname": "ProPlayer",
  "calificacion": 5,
  "comentario": "Excelente juego educativo, aprendí mucho recogiendo la basura",
  "fecha": "2026-04-23",
  "nivel_alcanzado": 3
}
```

Por qué estas valoraciones son importantes para tu entrega:

- [cite_start]**Representación de niveles**: Cubres los 4 niveles que mencionaste (Memorama, Secuencia, Basura y Conejo)[cite: 13].
- [cite_start]**Variedad de opiniones**: Usas diferentes calificaciones (3, 4 y 5 estrellas), lo que hace que los datos parezcan reales para un análisis[cite: 10].
- [cite_start]**Validación de datos**: Todos los campos numéricos (`id_usuario` , `calificacion` , `nivel_alcanzado`) están escritos sin comillas para que no te marque error de validación en MongoDB[cite: 8, 10, 13].
- [cite_start]**Consistencia**: Mantienes la estructura de `clave` que el profesor solicitó en el archivo original de ejemplo[cite: 1].